

30.07.20

## Algorithm Implementation

CH. Durga Prasad

192425305

MLA 0407

\* Design an MLP with an Input layer, two hidden layers, and an output layer. Implement the complete forward Propagation process to calculate the network output using MLP.

SO18 Aim:

To Implement the forward Propagation and Back Propagation algorithms for a multilayer perceptron (MLP) using Python and Numpy without using any deep learning libraries.

### Algorithm

#### Forward Propagation:

1. Initialize weights and biases randomly.
2. Compute the weighted sum:

$$Z = Wx + b$$

3. Apply the activation function (Sigmoid).
4. Repeat for all hidden layers.

#### Back propagation

1. Compute output layer error.
2. Calculate Gradients using the chain rule.
3. Propagate errors backward through hidden layers.
4. Compute Gradients for all weights and biases.
5. Update parameters using Gradient Descent.
6. Repeat for multiple epochs until convergence.

Code:

import numpy as np

def sigmoid(x):

return 1 / (1 + np.exp(-x))

def sigmoid\_derivative(x):

return x \* (1 - x)

x = np.array([[0, 0],

[0, 1],

[1, 0],

[1, 1]])

y = np.array([[0],

[1],

[0],

[1]])

np.random.seed(1)

w1 = np.random.rand(2, 2)

b1 = np.zeros((1, 2))

Predicted = (predictions >= 0.5).astype(int)

Accuracy = np.mean(Predicted == y) \* 100

Print("In Accuracy: ", accuracy, "%")

Sample output:

Epoch : 0 Loss : 0.2516

Final Prediction : 95%

Epoch : 1000 Loss : 0.1458

Epoch : 2000 Loss : 0.1952

Epoch : 3000 Loss : 0.0584

[0, 1] → 0.943

[1, 0] → 0.945

[1, 1] → 0.062

Result:

The MLP was successfully trained using forward and backward propagation, and the loss decreased over epochs.